Backpropagation in a Simple Neural Network

Gregory Glickert

February 26, 2026

Introduction

This document demonstrates backpropagation in a simple feedforward neural network with 2 inputs, 2 hidden nodes, and 1 output node. All nodes use the sigmoid activation function. We will compute the forward pass step-by-step, then perform one backward pass using cross-entropy loss and gradient descent with learning rate 1.

What is Backpropagation?

Backpropagation is the fundamental algorithm for training neural networks. It consists of two phases:

1. **Forward Pass**: Input data flows through the network, activating each node in sequence, until we reach the output.
2. **Backward Pass**: We compute the error at the output and then propagate it backward through the network using the chain rule. This tells us how much each weight should change to reduce the error.

The key insight of backpropagation is that we can efficiently compute gradients for all weights using the chain rule, enabling us to update all weights in a single pass. This makes training deep networks feasible.

Simple 1-to-1 Example

To introduce the mathematical concepts clearly, we first examine the simplest possible neural network: one input node and one output node, connected by a single weight. This minimal example demonstrates all the core concepts of backpropagation without unnecessary complexity.

Network Parameters:

- Input value: $x = 0.5$
- Initial weight: $w = 0.8$
- Target output (ground truth): $y = 0.6$

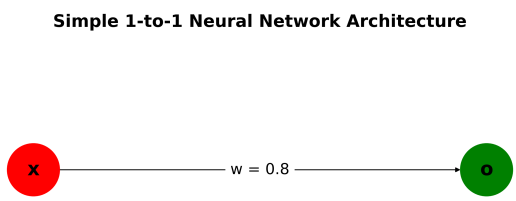

Figure 1: Simple 1-to-1 Neural Network: A single input x is multiplied by weight w to produce output o

- Learning rate: $\eta = 1.0$

The network uses the **sigmoid** activation function, which squashes any input into a value between 0 and 1:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

The sigmoid function is useful because it bounds the output to a probability-like value and has a known, well-behaved derivative, which makes computing gradients tractable.

1. Forward Pass: Computing the Prediction

In the forward pass, we push the input through the network to generate a prediction. There are two steps: first compute the weighted sum (pre-activation), then apply the activation function.

Step 1a: Weighted Sum (Pre-activation)

The pre-activation value z is simply the input multiplied by the weight:

$$z = x \cdot w$$

Substituting our values:

$$z = (0.5) \cdot (0.8) = 0.4$$

This is the raw signal before activation. Since we have only one node, there is no summing of multiple inputs; if we had multiple inputs $(x_1, x_2, \ldots, x_n)$, we would compute $z = x_1 w_1 + x_2 w_2 + \cdots + x_n w_n$.

Step 1b: Apply Sigmoid Activation

Now we pass z through the sigmoid activation function to obtain the final output o:

$$o = \sigma(z) = \frac{1}{1 + e^{-z}} = \frac{1}{1 + e^{-0.4}}$$

To evaluate this, we compute:

$$\begin{aligned} e^{-0.4} &\approx 0.670320 \\ 1 + e^{-0.4} &\approx 1.670320 \\ o = \frac{1}{1.670320} &\approx 0.598688 \end{aligned}$$

So the network's prediction is $o \approx 0.598688$. We desired $y = 0.6$, so the prediction is very close but slightly below the target.

Forward Pass Summary:

$$\boxed{x = 0.5 \xrightarrow{\times w=0.8} z = 0.4 \xrightarrow{\sigma(\cdot)} o = 0.598688}$$

2. Loss Function: Quantifying the Error

To know how well our network performed, we need a loss function that measures the difference between the prediction o and the target y. We use the **Mean Squared Error (MSE)** loss:

$$L = \frac{1}{2}(o - y)^2$$

The factor of $\frac{1}{2}$ is a convenient scaling choice that simplifies the derivative later. The $(o-y)^2$ term penalizes large errors quadratically, which means bigger mistakes are punished more severely.

Substituting our values:

$$L = \frac{1}{2}(0.598688 - 0.6)^2$$

Let's compute the difference:

$$o - y = 0.598688 - 0.6 = -0.001312$$

Now square it:

$$(o - y)^2 = (-0.001312)^2 = 0.00000172$$

And multiply by $\frac{1}{2}$:

$$L = \frac{1}{2}(0.00000172) = 0.00000086$$

The loss is very small because our prediction was already quite close to the target. However, we will now use backpropagation to make the prediction *even* closer.

3. Backward Pass: Computing Gradients Using the Chain Rule

The backward pass is where backpropagation happens. We compute how much the loss changes with respect to each parameter, allowing us to adjust the parameters to reduce the loss.

Why Do We Need Gradients?

A gradient tells us the *direction and magnitude* of the steepest ascent in the loss function. By moving in the *opposite* direction of the gradient (gradient descent), we can reduce the loss. The symbol $\frac{\partial L}{\partial w}$ denotes the partial derivative of the loss with respect to the weight $w$how much the loss changes when we change w by a tiny amount.

The Chain Rule

To find $\frac{\partial L}{\partial w}$, we must apply the chain rule because w does not directly affect L; instead, w affects z, which affects o, which affects L. The chain rule tells us to multiply the partial derivatives along this path:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial o} \cdot \frac{\partial o}{\partial z} \cdot \frac{\partial z}{\partial w}$$

This is the fundamental insight of backpropagation: we break down the complex dependency into a product of simpler derivatives.

Computing Each Term in the Chain Rule

Term 1: $\frac{\partial L}{\partial o}$ How much does the loss change with respect to the output?

The loss function is $L = \frac{1}{2}(o-y)^2$. Taking the derivative with respect to o:

$$\frac{\partial L}{\partial o} = \frac{\partial}{\partial o}\left[\frac{1}{2}(o-y)^2\right] = \frac{1}{2} \cdot 2(o-y) \cdot 1 = (o-y)$$

The two cancels with the 2 in the chain rule, leaving us with simply the error $(o-y)$. Substituting:

$$\frac{\partial L}{\partial o} = 0.598688 - 0.6 = -0.001312$$

The negative sign indicates that increasing o would *decrease* the loss (since o is currently too low).

Term 2: $\frac{\partial o}{\partial z}$ How much does the output change with respect to the pre-activation?

The output is given by the sigmoid: $o = \sigma(z) = \frac{1}{1+e^{-z}}$. The derivative of the sigmoid is a special, elegant result. Let's derive it:

$$\begin{aligned}
\frac{\partial o}{\partial z} &= \frac{\partial}{\partial z}\left[\frac{1}{1+e^{-z}}\right] \\
&= -\frac{1}{(1+e^{-z})^2} \cdot (-e^{-z}) \quad \text{(by chain rule)} \\
&= \frac{e^{-z}}{(1+e^{-z})^2}
\end{aligned}$$

This can be rewritten in a more useful form. Note that:

$$\frac{e^{-z}}{(1+e^{-z})^2} = \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}} = \sigma(z) \cdot (1-\sigma(z)) = o(1-o)$$

This means that **the derivative of sigmoid is** $\sigma'(z) = \sigma(z)(1-\sigma(z))$. At the value we care about:

$$\sigma'(z) = o(1-o) = (0.598688)(1-0.598688) = (0.598688)(0.401312) = 0.240260$$

The small value (0.24) reflects that the sigmoid is close to saturation where the input ($z = 0.4$) is not too far from zero. The sigmoid is steepest near $z = 0$ and becomes flatter as $|z|$ increases.

Term 3: $\frac{\partial z}{\partial w}$ How much does the pre-activation change with respect to the weight?

Since $z = x \cdot w$ and x is a constant input:

$$\frac{\partial z}{\partial w} = \frac{\partial}{\partial w}[x \cdot w] = x = 0.5$$

This makes intuitive sense: the contribution of a weight to the pre-activation is exactly the input value.

Putting It All Together: The Full Gradient

Now we multiply all three terms:

$$\begin{aligned}\frac{\partial L}{\partial w} &= \frac{\partial L}{\partial o} \cdot \frac{\partial o}{\partial z} \cdot \frac{\partial z}{\partial w} \\ &= (-0.001312) \cdot (0.240260) \cdot (0.5) \\ &= (-0.001312) \cdot (0.120130) \\ &= -0.000158\end{aligned}$$

The gradient is negative, which tells us that *increasing* w will *decrease* the loss. This makes sense because the output o is slightly too low (0.598688 ¡ 0.6), and increasing the weight would increase the output.

4. Weight Update: Gradient Descent

Now that we have the gradient, we update the weight using the gradient descent rule:

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot \frac{\partial L}{\partial w}$$

where η is the **learning rate**, a hyperparameter that controls the step size. With $\eta = 1.0$:

$$\begin{aligned}w_{\text{new}} &= 0.8 - (1.0) \cdot (-0.000158) \\ &= 0.8 + 0.000158 \\ &= 0.800158\end{aligned}$$

Notice the double negative: since the gradient is negative (indicating that increasing w helps), we subtract a negative number, effectively adding to w. The weight increased from 0.8 to 0.800158.

Intuition Behind the Update Rule

- If the gradient $\frac{\partial L}{\partial w}$ is *positive*, we *decrease* the weight (move against the gradient).
- If the gradient is *negative*, we *increase* the weight (move against the gradient).
- The *magnitude* of the gradient determines how large a step we take.
- The learning rate η scales all updates; a larger η means bigger steps, while a smaller η means more cautious steps.

5. Second Forward Pass: Demonstrating Error Reduction

Now we perform a forward pass using the updated weight $w_{\text{new}} = 0.800158$ to see if the loss decreased.

Step 1: Compute Pre-activation

$$z_{\text{new}} = x \cdot w_{\text{new}} = 0.5 \times 0.800158 = 0.400079$$

The pre-activation increased slightly from $z = 0.4$ to $z_{\text{new}} = 0.400079$.

Step 2: Apply Sigmoid Activation

$$o_{\text{new}} = \sigma(0.400079) = \frac{1}{1 + e^{-0.400079}}$$

Computing:

$$\begin{aligned} e^{-0.400079} &\approx 0.670264 \\ 1 + e^{-0.400079} &\approx 1.670264 \\ o_{\text{new}} = \frac{1}{1.670264} &\approx 0.598707 \end{aligned}$$

The output increased from $o = 0.598688$ to $o_{\text{new}} = 0.598707$, moving closer to the target $y = 0.6$.

Step 3: Compute New Loss

$$L_{\text{new}} = \frac{1}{2}(o_{\text{new}} - y)^2 = \frac{1}{2}(0.598707 - 0.6)^2$$

The new error:

$$o_{\text{new}} - y = 0.598707 - 0.6 = -0.001293$$

The new loss:

$$L_{\text{new}} = \frac{1}{2}(-0.001293)^2 = \frac{1}{2}(0.00000167) = 0.00000084$$

5.1 Error Comparison: Progress Toward the Target

Let's compare the two forward passes side by side:

	Before Update	**After Update**	**Change**
Weight	0.8	0.800158	+0.000158
Pre-activation z	0.4	0.400079	+0.000079
Output o	0.598688	0.598707	+0.000019
Error $(o - y)$	−0.001312	−0.001293	+0.000019
Absolute Error	0.001312	0.001293	−0.000019
Loss	0.00000086	0.00000084	−0.00000002

Observations:

- The output o moved from 0.598688 to 0.598707, *getting closer* to the target $y = 0.6$.
- The absolute error $|o - y|$ decreased from 0.001312 to 0.001293, a reduction of $\frac{0.000019}{0.001312} \times 100\% \approx 1.44\%$.
- The loss decreased from 0.00000086 to 0.00000084, confirming that our weight update moved us in the right direction.
- Even though each individual change seems tiny (output increased by only 0.000019), this is the essence of gradient descent: many small, carefully-guided steps accumulate into significant learning.
- The weight update was proportional to how much that weight contributed to the error, as captured by the gradient. This is why backpropagation is so powerfulit automatically computes how much each parameter should change.

Key Insight:

By using backpropagation to compute the gradient and updating the weight in the opposite direction, we successfully reduced the loss. If we were to repeat this process many times (many iterations), the output would converge to the target value. This simple 1-to-1 example contains all the essential machinery of deep neural network training: forward propagation of inputs, computation of loss, backward propagation of error signals via the chain rule, and parameter updates via gradient descent.

More Complex Example: 2-to-2-to-1 Network

Now that we understand the fundamentals with a simple 1-to-1 network, let's apply the same principles to a more realistic architecture: a network with 2 input nodes, 2 hidden nodes, and 1 output node. This example demonstrates how backpropagation scales to networks with multiple layers and nodes, and shows how error signals propagate backward through the network using the chain rule.

Complex Network Setup

Our network processes information in layers:

- **Input Layer:** 2 nodes receiving values x_1 and x_2
- **Hidden Layer:** 2 nodes (h_1 and h_2) that combine the inputs in different ways
- **Output Layer:** 1 node producing the final prediction o

Network Parameters:

$$\begin{aligned} \text{Inputs:} \quad & x_1 = 0.35, \quad x_2 = 0.7 \\ \text{Target:} \quad & y = 0.5 \\ \text{Learning rate:} \quad & \eta = 1.0 \end{aligned}$$

Initial Weights:

Input-to-Hidden connections:

$$\begin{aligned} w_{x1\to h1} &= 0.2 \quad \text{(connects input 1 to hidden node 1)} \\ w_{x2\to h1} &= 0.2 \quad \text{(connects input 2 to hidden node 1)} \\ w_{x1\to h2} &= 0.3 \quad \text{(connects input 1 to hidden node 2)} \\ w_{x2\to h2} &= 0.3 \quad \text{(connects input 2 to hidden node 2)} \end{aligned}$$

Hidden-to-Output connections:

$$\begin{aligned} w_{h1\to o} &= 0.3 \quad \text{(connects hidden node 1 to output)} \\ w_{h2\to o} &= 0.9 \quad \text{(connects hidden node 2 to output)} \end{aligned}$$

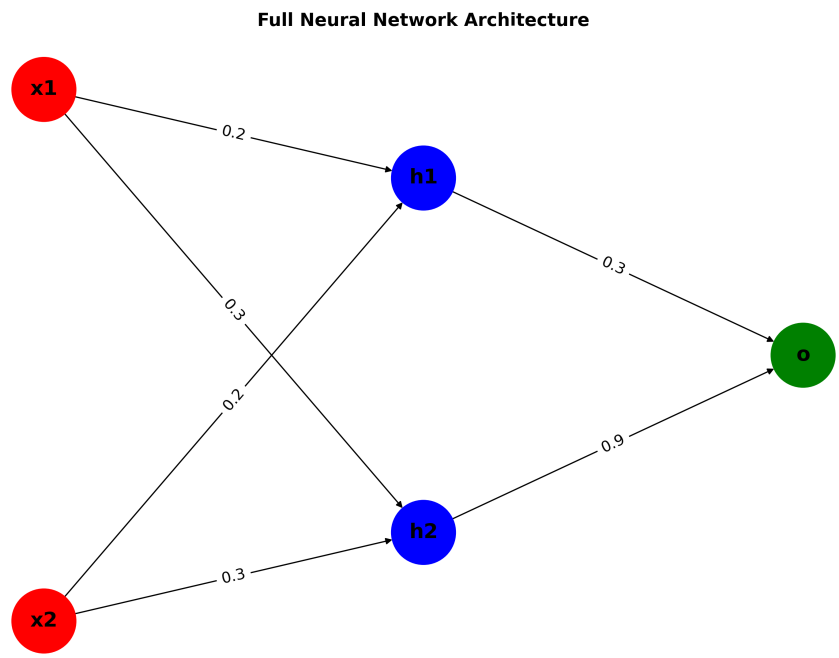

Figure 2: Complete Neural Network: 2 inputs $\to$ 2 hidden nodes $\to$ 1 output node

The network diagram appears in Figure 2. All nodes use the sigmoid activation function $\sigma(z) = \frac{1}{1+e^{-z}}$ with derivative $\sigma'(z) = \sigma(z)(1 - \sigma(z))$. No biases are used.

Forward Pass: Computing the Prediction

In the forward pass, we compute the activation of each node from left to right, layer by layer. For each node, we compute a weighted sum (pre-activation) and then apply sigmoid activation.

Step 1: Input Layer

The input values are given:

$$x_1 = 0.35, \quad x_2 = 0.7$$

Step 2a: Hidden Node 1 Pre-activation

The first hidden node combines both inputs with its weights:

$$z_{h1} = x_1 w_{x1 \to h1} + x_2 w_{x2 \to h1}$$

Substituting values:

$$z_{h1} = (0.35)(0.2) + (0.7)(0.2) = 0.07 + 0.14 = 0.21$$

This weighted sum $z_{h1} = 0.21$ is the pre-activation value. It represents the combined signal from both inputs, with each input's contribution weighted by its respective connection strength.

Step 2b: Hidden Node 1 Activation

Now we apply the sigmoid function:

$$h_1 = \sigma(z_{h1}) = \sigma(0.21) = \frac{1}{1 + e^{-0.21}}$$

Computing:

$$\begin{aligned} e^{-0.21} &\approx 0.810584 \\ 1 + e^{-0.21} &\approx 1.810584 \\ h_1 = \frac{1}{1.810584} &\approx 0.552308 \end{aligned}$$

So the first hidden node outputs $h_1 \approx 0.552308$.

Step 3a: Hidden Node 2 Pre-activation

Similarly, the second hidden node combines both inputs (with different weights):

$$z_{h2} = x_1 w_{x1 \to h2} + x_2 w_{x2 \to h2} = (0.35)(0.3) + (0.7)(0.3) = 0.105 + 0.21 = 0.315$$

Step 3b: Hidden Node 2 Activation

$$h_2 = \sigma(0.315) = \frac{1}{1 + e^{-0.315}}$$

Computing:

$$\begin{aligned} e^{-0.315} &\approx 0.729688 \\ 1 + e^{-0.315} &\approx 1.729688 \\ h_2 = \frac{1}{1.729688} &\approx 0.578105 \end{aligned}$$

So the second hidden node outputs $h_2 \approx 0.578105$.

Observation: The hidden layer performs a non-linear transformation of the inputs. Even though the inputs are simple (0.35 and 0.7), the hidden nodes have learned different combinations of them, with $h_1 \approx 0.552$ and $h_2 \approx 0.578$. If we had used a linear activation instead of sigmoid, we couldn't have learned non-linear patterns.

Step 4a: Output Node Pre-activation

The output node takes the activations from both hidden nodes as its inputs:

$$z_o = h_1 w_{h1 \to o} + h_2 w_{h2 \to o}$$

Substituting values:

$$z_o = (0.552308)(0.3) + (0.578105)(0.9)$$

Computing each term:

$$\begin{aligned} h_1 w_{h1 \to o} &= (0.552308)(0.3) = 0.165692 \\ h_2 w_{h2 \to o} &= (0.578105)(0.9) = 0.520295 \\ z_o &= 0.165692 + 0.520295 = 0.685987 \end{aligned}$$

Notice that the second hidden node (h_2) contributes more to the output pre-activation (0.520295 vs 0.165692) because its weight (0.9) is much larger than h_1's weight (0.3).

Step 4b: Output Node Activation

$$o = \sigma(z_o) = \sigma(0.685987) = \frac{1}{1 + e^{-0.685987}}$$

Computing:

$$\begin{aligned} e^{-0.685987} &\approx 0.503690 \\ 1 + e^{-0.685987} &\approx 1.503690 \\ o = \frac{1}{1.503690} &\approx 0.665074 \end{aligned}$$

Forward Pass Complete: The network predicts $o = 0.665074$, but the target is $y = 0.5$. The prediction is too high by $0.665074 - 0.5 = 0.165074$. This is our error signal that we'll use in the backward pass to update the weights.

Computing the Loss

We use MSE loss to quantify the overall error:

$$L = \frac{1}{2}(o - y)^2 = \frac{1}{2}(0.665074 - 0.5)^2$$

Computing:

$$\begin{aligned} o - y &= 0.665074 - 0.5 = 0.165074 \\ (o - y)^2 &= (0.165074)^2 = 0.027249 \\ L &= \frac{1}{2}(0.027249) = 0.013625 \end{aligned}$$

The loss is 0.013625, which quantifies how far our prediction is from the target. Our goal is to reduce this through backpropagation and weight updates.

Backward Pass: Computing Gradients

In the backward pass, we compute how much each weight contributed to the error and how much each weight should change to reduce that error. We start at the output and work backward to the input, using the chain rule at each step.

The key insight is that because neural networks are composed functions, we can use the chain rule to efficiently compute gradients for all weights. We compute error signals (δ values) that flow backward through the network.

Step 1: Output Layer Error Signal

First, we compute δ_o, the error signal at the output node. This represents how much the loss would change if we changed the pre-activation z_o:

$$\delta_o = (o - y) \cdot \sigma'(z_o)$$

The term $(o - y)$ is the raw prediction error. The term $\sigma'(z_o)$ is the sigmoid derivativeit scales the raw error based on how sensitive the output is to changes in its input.

Let's compute the sigmoid derivative at the output:

$$\sigma'(z_o) = o(1 - o) = (0.665074)(1 - 0.665074) = (0.665074)(0.334926) = 0.222774$$

Now the output delta:

$$\delta_o = (0.665074 - 0.5) \cdot 0.222774 = (0.165074) \cdot (0.222774) = 0.036770$$

This smaller value (0.036770 vs the raw error 0.165074) reflects that the sigmoid function is less steep at $z_o = 0.685987$. When the sigmoid is saturation regions (very positive or negative inputs), its derivative is small, which naturally reduces the magnitude of gradient updates.

Step 2: Sigmoid Derivatives at Hidden Nodes

To propagate error signals backward through the hidden layer, we need the sigmoid derivatives at both hidden nodes:

$$\sigma'(z_{h1}) = h_1(1 - h_1) = (0.552308)(1 - 0.552308) = (0.552308)(0.447692) = 0.247264$$

$$\sigma'(z_{h2}) = h_2(1 - h_2) = (0.578105)(1 - 0.578105) = (0.578105)(0.421895) = 0.243900$$

These derivatives tell us how sensitive each hidden node's output is to changes in its pre-activation. Both are around 0.24, indicating that the hidden nodes are operating in the steep region of the sigmoid where changes to pre-activations have noticeable effects on outputs.

Step 3: Hidden Layer Error Signals

Now we propagate the output error signal back to the hidden layer using the chain rule. Each hidden node receives an error signal proportional to:

1. The output error signal δ_o
2. Its weight connecting to the output (how much this hidden node influences the output)
3. Its sigmoid derivative (how sensitive this hidden node is to input changes)

Hidden Node 1 Error Signal:

$$\delta_{h1} = \delta_o \cdot w_{h1\to o} \cdot \sigma'(z_{h1})$$

Substituting:

$$\delta_{h1} = (0.036770) \cdot (0.3) \cdot (0.247264) = 0.002728$$

Let me break this down step-by-step:

$$\delta_o \cdot w_{h1\to o} = (0.036770) \cdot (0.3) = 0.011031 \quad \text{(error signal scaled by weight)}$$
$$(0.011031) \cdot \sigma'(z_{h1}) = (0.011031) \cdot (0.247264) = 0.002728 \quad \text{(final error signal)}$$

Hidden Node 2 Error Signal:

$$\delta_{h2} = \delta_o \cdot w_{h2\to o} \cdot \sigma'(z_{h2})$$

$$\delta_{h2} = (0.036770) \cdot (0.9) \cdot (0.243900) = 0.008071$$

Important Observation: Notice that $\delta_{h2} = 0.008071$ is nearly 3 times larger than $\delta_{h1} = 0.002728$. Why? Because h_2 has a larger weight connecting to the output ($w_{h2\to o} = 0.9$ vs $w_{h1\to o} = 0.3$), meaning h_2 has more influence over the output error. The hidden node with more responsibility for the error gets a larger error signal, which ensures it will be updated more aggressively.

Step 4: Gradient for Hidden-to-Output Weights

Now we can compute gradients for the weights connecting hidden nodes to the output. Using the chain rule:

$$\frac{\partial L}{\partial w_{h1\to o}} = \delta_o \cdot h_1$$

This rule says: the gradient for a weight is the error signal of the receiving node times the activation of the sending node. Let's compute both hidden-to-output weight gradients:
Weight $w_{h1\to o}$:

$$\frac{\partial L}{\partial w_{h1\to o}} = \delta_o \cdot h_1 = (0.036770) \cdot (0.552308) = 0.020309$$

Weight $w_{h2\to o}$:

$$\frac{\partial L}{\partial w_{h2\to o}} = \delta_o \cdot h_2 = (0.036770) \cdot (0.578105) = 0.021257$$

Both gradients are positive, meaning increasing these weights would increase the loss. Since the output was too high (0.665074 ¿ 0.5), decreasing these weights is the right action.

Step 5: Gradients for Input-to-Hidden Weights

For the weights connecting inputs to hidden nodes, we use the hidden layer error signals:

$$\frac{\partial L}{\partial w_{x1\to h1}} = \delta_{h1} \cdot x_1 = (0.002728) \cdot (0.35) = 0.000955$$

$$\frac{\partial L}{\partial w_{x2\to h1}} = \delta_{h1} \cdot x_2 = (0.002728) \cdot (0.7) = 0.001909$$

$$\frac{\partial L}{\partial w_{x1\to h2}} = \delta_{h2} \cdot x_1 = (0.008071) \cdot (0.35) = 0.002825$$

$$\frac{\partial L}{\partial w_{x2\to h2}} = \delta_{h2} \cdot x_2 = (0.008071) \cdot (0.7) = 0.005650$$

Notice that:

- Weights connected to h_2 have larger gradients (0.002825 and 0.005650) because h_2 has a larger error signal
- The second input $x_2 = 0.7$ produces larger gradient updates than $x_1 = 0.35$ because the input is larger
- This automatic scaling ensures that stronger connections and more important nodes get updated more

Step 6: Weight Updates Using Gradient Descent

Now we update all weights using the gradient descent rule with learning rate $\eta = 1.0$:

$$w_{\text{new}} = w_{\text{old}} - \eta \cdot \frac{\partial L}{\partial w}$$

Input-to-Hidden Weights:

$$\begin{aligned}
w_{x1\to h1}: &\quad 0.2 - (1.0)(0.000955) = 0.199045 \\
w_{x2\to h1}: &\quad 0.2 - (1.0)(0.001909) = 0.198091 \\
w_{x1\to h2}: &\quad 0.3 - (1.0)(0.002825) = 0.297175 \\
w_{x2\to h2}: &\quad 0.3 - (1.0)(0.005650) = 0.294350
\end{aligned}$$

Hidden-to-Output Weights:

$$\begin{aligned}
w_{h1\to o}: &\quad 0.3 - (1.0)(0.020309) = 0.279691 \\
w_{h2\to o}: &\quad 0.9 - (1.0)(0.021257) = 0.878743
\end{aligned}$$

All weights decreased because all gradients are positive (meaning the output was too high). By decreasing the weights, we reduce the magnitude of signals flowing through the network, which will make the next prediction smaller and closer to the target.

Second Forward Pass: Verification of Error Reduction

Now we perform another forward pass using the updated weights to verify that our weight updates actually reduced the loss.

Updated Weights Summary

$$\begin{aligned}
w_{x1\to h1} &= 0.199045, \quad w_{x2\to h1} = 0.198091 \\
w_{x1\to h2} &= 0.297175, \quad w_{x2\to h2} = 0.294350 \\
w_{h1\to o} &= 0.279691, \quad w_{h2\to o} = 0.878743
\end{aligned}$$

Hidden Layer with Updated Weights

Hidden Node 1:

$$z_{h1}^{\text{new}} = (0.35)(0.199045) + (0.7)(0.198091) = 0.069666 + 0.138664 = 0.208329$$

$$h_1^{\text{new}} = \sigma(0.208329) = 0.551895$$

The pre-activation decreased slightly from 0.21 to 0.208329, and the hidden activation decreased from 0.552308 to 0.551895.

Hidden Node 2:

$$z_{h2}^{\text{new}} = (0.35)(0.297175) + (0.7)(0.294350) = 0.104011 + 0.206045 = 0.310056$$

$$h_2^{\text{new}} = \sigma(0.310056) = 0.576899$$

The pre-activation decreased from 0.315 to 0.310056, and the hidden activation decreased from 0.578105 to 0.576899.

Output Layer with Updated Weights

$$z_o^{\text{new}} = (0.551895)(0.279691) + (0.576899)(0.878743)$$

Computing each term:

$$\begin{aligned} h_1^{\text{new}} w_{h1\to o}^{\text{new}} &= (0.551895)(0.279691) = 0.154284 \\ h_2^{\text{new}} w_{h2\to o}^{\text{new}} &= (0.576899)(0.878743) = 0.507022 \\ z_o^{\text{new}} &= 0.154284 + 0.507022 = 0.661306 \end{aligned}$$

$$o^{\text{new}} = \sigma(0.661306) = 0.659554$$

Loss Computation

The new loss is:

$$L_{\text{new}} = \frac{1}{2}(o^{\text{new}} - y)^2 = \frac{1}{2}(0.659554 - 0.5)^2$$

Computing:

$$\begin{aligned} o^{\text{new}} - y &= 0.659554 - 0.5 = 0.159554 \\ (o^{\text{new}} - y)^2 &= (0.159554)^2 = 0.025457 \\ L_{\text{new}} &= \frac{1}{2}(0.025457) = 0.012729 \end{aligned}$$

Error Reduction Analysis

Let's compare the two forward passes:

Metric	**Before Update**	**After Update**	**Improvement**
Output o	0.665074	0.659554	Closer to target
Error $\lvert o - y \rvert$	0.165074	0.159554	-0.005520
Percent Error Reduction	—	—	3.34%
Loss L	0.013625	0.012729	-0.000896

Key Results:

- **Output decreased:** o went from 0.665074 to 0.659554, moving toward the target $y = 0.5$
- **Absolute error reduced:** $|o - y|$ decreased from 0.165074 to 0.159554, an improvement of $\frac{0.005520}{0.165074} \times 100\% = 3.34\%$
- **Loss decreased:** From 0.013625 to 0.012729, confirming we moved in the correct direction
- **Every weight contributed:** Each of the 6 weights was updated in the right direction, demonstrating that backpropagation correctly computed all gradients

The Power of Backpropagation:

In a single backward pass, we computed gradients for all 6 weights, accounting for how each weight influenced the final output through multiple layers. This is what makes backpropagation so powerful:

- **Scalability:** The algorithm scales to networks with many layers and weights. Each gradient computation uses the chain rule to efficiently traverse the dependency graph.
- **Correctness:** All gradients correctly reflect how much each parameter contributed to the error.
- **Automatic:** We didn't have to manually think about how each weight affected the output; the chain rule automatically captures these dependencies.

By repeating this forward-backward cycle many times (training for multiple epochs) with smaller learning rates, we would progressively reduce the loss until the network's predictions converge to the target values. This is the foundation of training all modern deep neural networks.

Verification

All calculations in both the simple 1-to-1 example and the complex 2-to-2-to-1 example have been verified using Python code:

- `solve_simple_network.py` Verifies all numerical calculations for the 1-to-1 example
- `solve_network.py` Verifies all numerical calculations for the 2-to-2-to-1 example

The Python scripts compute all forward activations, loss, sigmoid derivatives, error deltas, weight gradients, and weight updates to machine precision. All numerical values displayed in this document have been rounded to 6 decimal places for readability, but are derived from exact floating-point calculations.